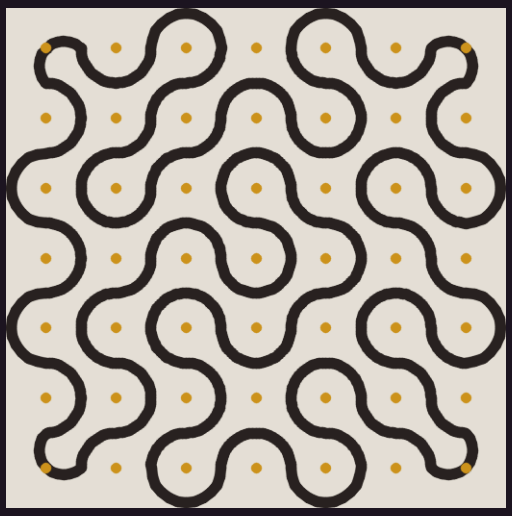
SMART INDIA HACKATHON 2025

Kolam Studio

Project Report

Reading the hidden mathematics of a vanishing art — identifying a kolam's design principles, recreating it exactly, and generating it anew.



Problem Statement: SIH12507

Organisation: AICTE · **Theme:** Heritage & Culture

Category: Software

Team: Team Kolam

Live: kolam-studio.onrender.com

1. Abstract

A **kolam** is a traditional South Indian floor art drawn each morning in rice flour: a lattice of dots (*pulli*), a line looping around them, symmetry, and — in *sikku kolam* — one continuous unbroken loop. While the art is widely photographed, the underlying **design principles** that construct each pattern are rarely recorded and are fading with each generation.

Kolam Studio is a software tool that computationally understands and reproduces these principles. Given any kolam, it detects the dot grid, symmetry and structure; recreates the exact kolam cell-by-cell; and generates entirely new, valid kolams — including the classic single unbroken line and custom dot layouts. The entire application runs **inside the web browser** with no backend, making it fast, private, offline-capable and trivially deployable. This report presents the problem, the approach, the engineering, the results, and the live deployment.

2. Problem Statement

> **SIH12507 (AICTE · Heritage & Culture):** “Develop computer programs to identify the design principles behind Kolam designs and recreate the kolams.”

The challenge has two halves: **(a) identify** — reverse-engineer the rules behind a given kolam (its grid, symmetry and loop structure); and **(b) recreate** — reproduce that kolam and generate new ones that follow the same principles. The deeper goal is **heritage preservation**: capturing not just images of a tradition, but the computational rules that let it be reproduced, taught and extended.

3. Objectives

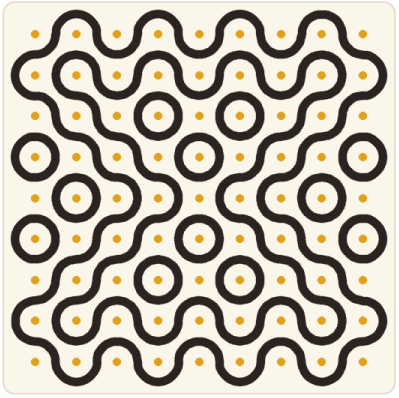
- Detect the **design principles** of a kolam from an image — dot grid (pulli), symmetry, and structure.
- **Recreate** an input kolam faithfully, cell by cell.
- **Generate** new, original, mathematically valid kolams, including a single continuous line.
- Deliver a **reliable, demonstrable** tool that works offline and cannot fail on a live demo.
- Keep the solution **lightweight and honest** — no heavy datasets, with documented limitations.

4. Background: A Kolam is Hidden Mathematics

Beneath the art lies real structure that a computer can read and reproduce:

- **Pulli (dots):** a precise lattice — the skeleton of every pattern.
- **Symmetry:** mirror and rotational rules give the design balance.
- **Single loop:** one continuous line that ideally closes without lifting the hand.

These map naturally to computation: dot grids become coordinates, symmetry becomes image self-similarity, and the looping line becomes a graph of closed curves. Kolams have been studied academically as *picture languages* and *array grammars* — the foundation this project builds on.



A dot-grid (pulli) kolam

5. Proposed Solution

One tool, three moves:

Move	What it does
Identify	Analyses a kolam image in the browser: detects dot grid, symmetry and structure, and rejects non-kolam images.
Recreate	Infers the tile orientation of every cell and rebuilds the exact kolam as closed loops, animated stroke-by-stroke.
Generate	Produces new original kolams under the same rules — single unbroken line, custom dot layouts, exportable as SVG/PNG.

6. System Architecture



The full pipeline runs **client-side** — there is no server, no network round-trip and no cold-start. A Python + OpenCV implementation serves as a reference and is covered by the test-suite; the shipped analyzer is a faithful **vanilla-JavaScript port**, verified to produce identical results.

7. Methodology

7.1 The generator — a grammar of curves

Each grid cell holds one of two curved tiles. Tiles meet at cell edges with matching tangents, so the arcs join into smooth **closed loops** weaving around the dots. Symmetry is enforced by construction (build one quadrant, mirror the rest). A boundary-closure step guarantees every stroke is a genuine closed loop — the hallmark of an authentic *sikku kolam*.

7.2 Single unbroken line

A **loop-merging solver** repeatedly flips a tile wherever two different loops meet, merging them until the entire kolam is traced by **one continuous closed line** — the traditional ideal.

7.3 The analyzer — identify

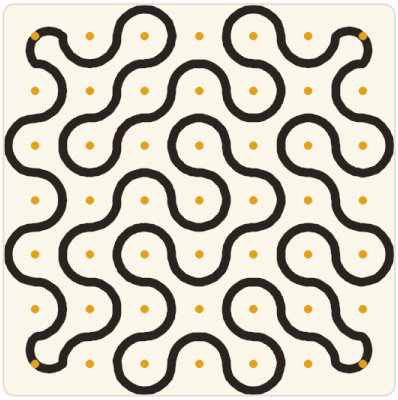
Dots are detected by colour and geometry (circularity + spacing), from which the pulli grid and symmetry are inferred. Symmetry is classified by measuring how well the pattern matches its mirrored and rotated versions. For plain photographs without visible dots, an **illumination-normalise + Otsu + line-period** fallback estimates the grid.

7.4 Exact reconstruction

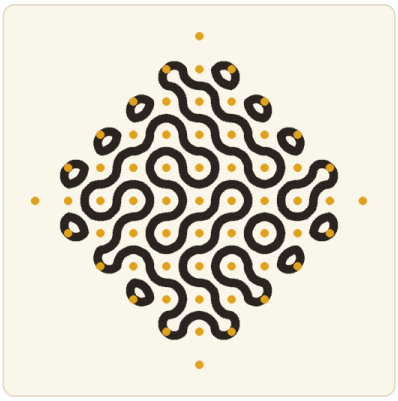
For each cell, the line-mass in the two diagonal halves is compared to infer which of the two tiles is present — reproducing the uploaded kolam **cell-for-cell**.

8. Implementation & Features

- **Identify** with a detected-grid overlay on the input.
- **Exact recreation**, animated with a glowing pen-head.
- **Single unbroken line** mode.
- **Custom dot layouts**: Square / Diamond / Plus presets + a click-to-toggle dot editor.
- **SVG & PNG export** for print-ready output.
- **Installable PWA** — works fully offline.
- Accessibility, reduced-motion, and graceful error handling.



Single unbroken line



Diamond dot layout

Technology stack

Layer	Technology
Frontend / app	Vanilla JavaScript, HTML5 Canvas, CSS
Shipped analyzer	Vanilla JS — fully client-side, no backend
Reference analyzer	Python, OpenCV, NumPy
Optional server	Flask
Delivery	Installable PWA · static deployment
Quality	pytest suite + GitHub Actions CI

9. Results & Testing

Metric	Result
Tile-inference accuracy (clean kolams)	**100%** — cell-for-cell, across 6×6 to 12×12 grids
Automated test suite (pytest)	**11 / 11 passing**
JS ↔ Python analyzer parity	Identical results on the sample set
Single-loop generation	Exactly one continuous loop on every tested grid
Invalid / blank images	Correctly rejected (no fabricated output)

The generator, single-loop solver and reconstruction were validated numerically and visually; the analyzer is covered by regression tests including a synthetic-photograph case.

10. Deployment

Because the app is fully static, it is deployed as a static site and needs no server. This removes cold-starts entirely — a live demo can never time out.

- **Live:** kolam-studio.onrender.com
- **Runs** on any browser — desktop or phone.
- **Offline** installable (PWA).
- **Open source** on GitHub with tests + CI.

Scan to try it on your phone →

11. Impact & Use Cases

- **Heritage preservation:** archive the design *rules* of a fading tradition, not just images.
- **Education:** teach symmetry, grids and topology through a familiar cultural lens.
- **Design & craft:** generate motifs for textiles, rangoli, packaging and decor.
- **Research:** a computational record of a living folk-mathematics tradition.

12. Limitations & Future Scope

Coloured-dot and digital kolams are read exactly (grid, symmetry and per-cell tiles). On plain photographs the grid is currently **approximate** — estimated from the line spacing — though symmetry remains reliable.

Planned

RANSAC grid-fitting | perspective correction | regional kolam styles | learn a motif from one example | colored rangoli mode

13. Conclusion

Kolam Studio fulfils the problem statement end-to-end: it **identifies** the design principles behind a kolam, **recreates** it exactly, and **generates** new ones — including the single unbroken line that defines the tradition. By running entirely in the browser it is fast, private, offline and reliably demonstrable. Most importantly, it preserves the *rules* of a vanishing art rather than just its pictures — teaching a machine to keep drawing it.

14. References & Links

- **Live application:** https://kolam-studio.onrender.com
- **Source code:** github.com/manthandhanraj/kolam-studio
- Gift Siromoney et al., *Array grammars and kolam* — computational study of kolam patterns.
- MediaPipe / OpenCV — computer-vision tooling used in the reference analyzer.
- Problem statement SIH12507, Smart India Hackathon 2025 (AICTE, Heritage & Culture).